Technical Architecture & Implementation Plan

Application-Level Data Encryption for Stored Communications

CONFIDENTIAL V1.0 PLANNING

System: Navriti RTC Communication Engine
Target: Apache Cassandra Multi-Tenant Storage
Date: September 2026

> **Executive Overview**
> This document details the architectural design and end-to-end implementation plan for applying authenticated application-level encryption (**AES-256-GCM**) to all sensitive communication data (messages, announcements, attachments) stored in Apache Cassandra. The solution preserves multi-tenant platform isolation (`platform_id`), eliminates raw plaintext exposure in database storage/backups, and guarantees zero breaking changes to existing client-side protocols or database schemas.

1. Current Architecture Summary

The Navriti RTC platform is powered by a Node.js/Express backend with Socket.IO for real-time signaling, persisting data to an 11-table Apache Cassandra keyspace (`navriti`). Every table is strictly partitioned by `platform_id` to guarantee tenant isolation.

```
[Client Web / Mobile Apps]
    │ (REST API & WebSockets wss://)
    ▼
[Express Controllers & Socket.IO Handler]
    │ (Plaintext JS Objects)
    ▼
[Repository / DAO Layer] ◄──► [Centralized EncryptionService] (AES-256-GCM)
    │ (Encrypted Envelopes enc:v1:iv:tag:cipher)
    ▼
[Apache Cassandra Cluster (11 Partitioned Tables)]
```

2. Feature-by-Feature Data Flow

2.1 Chat Messaging (1-on-1 & Group)

- **Write Flow:** Client sends message via REST or Socket (`message:send`). The repository encrypts the `content` and serialized `attachment` JSON into GCM envelopes (`enc:v1:...`) before executing concurrent CQL writes to `messages_by_conversation` and `messages_by_id` . Decrypted plaintext entities are returned in-memory to the socket handler to broadcast to room subscribers.
- **Read Flow:** On `getMessages()` or `findMessageById()` , Cassandra rows are fetched by partition key. The repository layer decrypts the envelopes on-the-fly and deserializes attachment JSON before returning clean objects to the client.

2.2 Announcement Portals & Broadcasts

- **Write Flow:** Admin publishes/updates announcements via REST or Socket. The repository encrypts `title` , `content` , and `attachments` JSON, saving them to `announcements_by_portal` and `announcements_by_id` .
- **Read Flow:** When users query portal feeds, rows are decrypted at the repository boundary and verified against audience criteria (`target_audience` , `target_user_ids`) before response delivery.

3. Exact Encryption Candidates

Table Name	Column	CQL Type	Stored Content Description	Reason for Encryption
`messages_by_conversation`	`content`	`text`	Raw chat message body (text, emojis, markdown)	Direct user communication; contains PII and confidential conversations.
`messages_by_conversation`	`attachment`	`text`	Serialized JSON containing Cloudinary secure URLs, media dimensions, filenames	Prevents exposure of private storage asset URLs and file names in database backups.
`messages_by_id`	`content`	`text`	Raw chat message body	Direct lookup copy; mirrors `messages_by_conversation`.
`messages_by_id`	`attachment`	`text`	Serialized media metadata JSON	Direct lookup copy; mirrors `messages_by_conversation`.
`announcements_by_portal`	`title`	`text`	Headline / Subject of announcement	Protects organizational news and confidential broadcasts.
`announcements_by_portal`	`content`	`text`	Full announcement body copy	Protects sensitive corporate or institutional notices.
`announcements_by_portal`	`attachments`	`text`	Serialized JSON array of file attachment URLs	Prevents leak of confidential documents and media links.
`announcements_by_id`	`title`, `content`, `attachments`	`text`	Headline, body copy, and media attachments	Direct lookup copy; mirrors `announcements_by_portal`.

4. Fields That Must Remain Plaintext

Column Name	CQL Type	Functional Role	Reason for Plaintext Retention
`platform_id`	`text`	Partition Key in all 11 tables	Required for multi-tenant Cassandra sharding and replica query routing.
`conversation_id` / `portal_id`	`text` / `uuid`	Partition Key	Primary lookup keys for conversation streams and announcement portals.
`message_id` / `announcement_id`	`uuid`	Clustering / Partition Key	Ensures unique row identification and deduplication.
`created_at` / `published_at`	`timestamp`	Clustering Key (`ORDER BY DESC`)	Enables chronological pagination (`LIMIT` , range queries). Encrypting breaks ordering.
`sender_id` / `user_id`	`text`	Foreign key / Lookup mapping	Required for access control validation, socket routing, and identity checks.
`target_audience` / `target_user_ids`	`text` / `set<text>`	Audience filtering attribute	Evaluated in CQL/application logic for authorization and audience distribution.
`message_type` , `status` , `is_deleted` , `edited`	`text` / `boolean`	State management flags	Required for protocol handling and conditional update queries.

5. Recommended Encryption Architecture

5.1 Cryptographic Primitive: Authenticated Symmetric Encryption (AES-256-GCM)

- **Algorithm:** `AES-256-GCM` (Galois/Counter Mode). Provides confidentiality (256-bit key) and integrity/authenticity (128-bit authentication tag). Detects tampering or bit-flipping at rest.
- **Initialization Vector (IV):** Cryptographically random 12-byte (96-bit) IV generated per field write via `crypto.randomBytes(12)`. Guarantees zero IV reuse.

5.2 Storage Envelope & Serialized Ciphertext Format

Encrypted data is packaged into a compact, self-describing ASCII string stored directly inside existing Cassandra `text` columns:

```
enc:<version>:<iv_base64>:<auth_tag_base64>:<ciphertext_base64>
```

Example Envelope:

```
enc:v1:qK8jZ1eL9Xb0A3d2:F8d2kW1sZ5mN9qX4p0R==:a8Kd01vMZ3p9wX81...==
```

6. Proposed Integration Points & DAO Boundary

- **Boundary Placement:** Integration is strictly placed inside `messageRepository.js` and `announcementRepository.js`.
- **Zero Controller / Socket Coupling:** Controllers and Socket handlers continue processing clean JS objects, completely unaware of database encryption.
- **Dual-Write Guarantee:** Both partition tables (e.g. `messages_by_conversation` and `messages_by_id`) receive synchronized encrypted payloads.

7. Cassandra & Schema Impact

- **Zero Schema Changes:** All encrypted candidate columns are already `text` type. No table rebuilds or DDL alterations needed.
- **Storage Overhead:** Adds ~45–55 bytes per field for Base64 IV and Auth Tag. SSTable block compression (LZ4) keeps disk overhead negligible.
- **Query Performance:** No impact on query speed because queries index by partition keys (`platform_id`, `conversation_id`), not textual content.

8. Real-Time & Socket Impact

- **Transport Layer:** Real-time WebSocket payloads remain protected in transit via TLS (`wss://`).
- **No Client-Side Changes:** Connected clients receive standard decrypted JSON events (`message:new`, `announcement:new`). No client app updates required.
- **Microsecond Latency:** AES-256-GCM executes in < 0.05ms per message via Node.js native OpenSSL bindings.

9. Backward Compatibility & Edge-Case Protection

Scenario	Risk / Failure Mode	Built-In Mitigation Strategy
Legacy Plaintext in DB	Decryption error when reading unencrypted historical rows.	**Safe Passthrough:** If `!val.startsWith('enc:')`, returns raw string untouched without error.
Null / Empty Fields	Crash on `crypto.createCipheriv` with null/undefined values.	Guard condition `if (!value) return value;` skips empty inputs cleanly.
Double Encryption on Edits	Double envelope wrapping when updating existing records.	Verification guard `if (isEncrypted(val)) return val;` prevents duplicate ciphers.
Tampered / Corrupted Data	GCM authentication tag failure causes worker crash.	Graceful `try/catch` returns fallback `"[Encrypted content unreadable]"` and logs security alert.

10. Migration & Key Extensibility Strategy

- **Phase 1 (Dual-Read & Encrypted-Write):** All new rows are written with `enc:v1:`; reads seamlessly handle both legacy plaintext and ciphertext.
- **Phase 2 (Background Backfill):** An offline background script scans partitions and upgrades legacy plaintext rows to `enc:v1:` without downtime.
- **Multi-Key Rotation:** Supporting `enc:v2:` by referencing key version identifiers in the envelope.
- **Platform-Derived Keys (HKDF):** Extensible to per-tenant cryptographic keys using `crypto.hkdfSync` derived from `platform_id`.

11. Comprehensive Testing Plan

- **Unit Tests:** Verify encrypt/decrypt symmetry, IV randomness, auth-tag tamper rejection, and legacy plaintext passthrough.
- **Repository Tests:** Verify raw Cassandra rows contain `enc:v1:` while repository return values contain decrypted plaintext.
- **Real-Time Socket Tests:** Verify WebSocket events deliver clean, unencrypted JSON payloads to client listeners.

12. Risk Assessment Matrix

Risk Category	Severity	Impact	Mitigation Strategy
Master Key Loss	**HIGH**	Irreversible data loss for encrypted records.	Store keys in enterprise secret managers (AWS Secrets Manager, Azure Key Vault, HashiCorp Vault).
Key Compromise	**MEDIUM**	Historical database dumps readable.	Rotate keys instantly by configuring `enc:v2:` envelope versioning.
Accidental Double Encryption	**MEDIUM**	Corrupted text on edits.	Enforce `isEncrypted()` checks before all encryption operations.
CPU Overhead	**LOW**	Minor CPU cycle increase.	Hardware-accelerated AES-NI ensures < 0.1% CPU utilization impact.